

Aarya Arban

T11 – 05

Assignment No. 10

Aim: Implement any machine learning techniques using available dataset to develop a recommendation system.

Theory:

A recommendation system is a type of information filtering system that predicts the preferences or ratings a user would give to a particular item. These systems are widely used in various online platforms such as e-commerce websites, streaming services, social media platforms, and more. Recommendation systems aim to provide personalised suggestions to users, thereby enhancing user experience and increasing engagement.

There are several types of recommendation systems, each with its own approach and methodology:

1. Content-Based Filtering:

Content-based filtering recommends items to users based on their previous actions or preferences. It analyzes the attributes or features of items that a user has interacted with in the past and recommends similar items.

Advantages:

It doesn't require data about other users, making it suitable for new users.

It can provide personalized recommendations based on the user's specific interests.

Disadvantages:

It tends to recommend items that are similar to what the user has already seen, potentially limiting serendipity.

It might not capture diverse interests or introduce users to new content.

2. Collaborative Filtering:

Collaborative filtering recommends items to users based on the preferences of other users. It identifies users with similar preferences and recommends items liked by those users.

Advantages:

It can capture complex patterns in user preferences.

It doesn't require detailed item attributes, making it applicable to various types of items.

Disadvantages:

It suffers from the cold start problem, where new items or users have insufficient data for accurate recommendations.

It can face scalability issues with large datasets and user bases.

3. Hybrid Recommendation Systems:

Hybrid recommendation systems combine multiple recommendation techniques, such as contentbased filtering and collaborative filtering, to provide more accurate and diverse recommendations.

Advantages:

It leverages the strengths of different recommendation approaches, potentially mitigating their individual weaknesses.

It can adapt to different user preferences and scenarios.

Disadvantages:

Designing and implementing hybrid systems can be complex and resource-intensive.

Integration of different techniques may introduce additional computational overhead.

4. Knowledge-Based Recommendation Systems:

Knowledge-based recommendation systems use explicit knowledge about items and user preferences to make recommendations. They often incorporate domain knowledge or constraints user to provide personalized suggestions.

Advantages:

- They can provide explanations for recommendations based on domain knowledge.

- They can handle cold start problems more effectively by relying on explicit user preferences.

Disadvantages:

- They require substantial domain knowledge or structured data, which may not always be available or easily accessible.

- They may struggle with scaling to large and diverse datasets.

Each type of recommendation system has its own set of advantages and disadvantages, and the choice of the most suitable type depends on factors such as the nature of the items, available data, user preferences, and system requirements. Additionally, advancements in machine learning and artificial intelligence continue to drive innovations in recommendation systems, leading to the development of more sophisticated and effective approaches.

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files
under the input directory

import os for dirname, _, filenames in os.walk('/kaggle/input'): for
filename in filenames: print(os.path.join(dirname, filename))

import pandas as pd import random

pd.set_option('display.max_columns', None) pd.set_option('display.width', 500)
pd.set_option('display.expand_frame_repr', False) from
sklearn.feature_extraction.text import TfidfVectorizer from
sklearn.metrics.pairwise import cosine_similarity

df = pd.read_csv("movies_metadata.csv", skiprows = lambda i: i % 4) df.head()

{"type":"dataframe","variable_name":"df"} df.shape
(11366, 24)
print(df.isnull().sum())
```

adult	0
belongs_to_collection	10258
budget	0 genres
0 homepage	9382 id
0 imdb_id	3
original_language	3
original_title	0 overview
238 popularity	1
poster_path	88
production_companies	0
production_countries	0
release_date	20 revenue
2	

```
runtime          68
spoken_languages 2 status
21 tagline       6241 title
2 video          2 vote_average
2 vote_count     2
```

```
sum()
```

```
dtype: int64
```

```

df.isnull().
adult 0
belongs_to_collection
10258 budget 0
genres 0
homepage 9382 id
0 imdb_id 3
original_language 3
original_title 0
overview 23
popularity
poster_path 8
production_companies
production_countries
runtime
release_date 2
revenue 2
dtype: int64
M = 2500 C = df[
def ( df["weighted_rat
df["vote_count" ascendi
{
    title 2 video
    vote_average 2
    vote_count 2 i

    'vote_average'].mean()
return (v /
weighted_rating(r, v, M, C):
(V) * r) + (M / (v + M) * C
    ], M, C) df.sort_value)
lse).head(10)

```

```
df["overview"].head()
```

```
0 Cheated on, mistreated and stepped on, the wom... 1 A  
mischievous young boy, Tom Sawyer, witnesses...
```

```
2 When a lawyer shows up at the vampire's doorst... 3 The life  
of the gambling paradise – Las Vegas ... 4 A vengeful New York  
transit cop decides to ste...
```

```
Name: overview, dtype: object
```

```
df = df[~df["title"].duplicated(keep='last')] df  
= df[~df["title"].isna()] df = df[~df["overview"].isna()]
```

```
tfidf = TfidfVectorizer(stop_words="english") tfidf_matrix =  
tfidf.fit_transform(df['overview']) tfidf_matrix =  
tfidf_matrix.astype(np.float32) tfidf_matrix.shape
```

```
(10882, 37304)
```

```
tfidf_matrix
```

```
<10882x37304 sparse matrix of type '<class 'numpy.float32'>'
```

```
with 298586 stored elements in Compressed Sparse Row format>
```

```
df['title'].shape
```

```
(10882,)
```

```
tfidf.get_feature_names_out()
```

```
array(['00', '000', '000th', ..., '찾', 'five'], '찾',  
'찾', ' ',  
dtype=object)  
'찾'
```

```
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)  
cosine_sim.shape
```

```
(10882, 10882)
```

```
cosine_sim[1]
```

```
array([0.00922137, 0.99999994, 0. , ..., 0. ,
```



```
0.01560123,
```

```
0.    ], dtype=float32)
```

```
indices = pd.Series(df.index, index=df['title'])
```

```
indices.index.value_counts().sort_values(ascending=False)
```

```
Waiting to Exhale      1
```

```
Twelve Monkeys        1
```

Vampire in Brooklyn	1	
8898Black Sheep		1
Last Summer in the Hamptons	1	
5012		
The Man with the Rubber Head	1	
The Untameable Whiskers	1	
The Fat and Lean Wrestling Match	1	
St. Michael Had a Rooster	1	
Betrayal	1	

Name: title, Length: 10882, dtype: int64

array([0., 0., 0., ..., 0., 0., 0.],
dtype=float32)indices["Cinderella"]

indices["Django
Unchained"]

*# Check if the
movie title
"Cinderella" is
present in the
index if*

"Cinderella" in

indices:
movie_index =

indices["Cinderella"]

cosine_sim_cinderella = cosine_sim[movie_index]

print(cosine_sim_cinderella) **else:** **print**("Movie title 'Cinderella' is not found in
the DataFrame index.")

[0. 0. 0. ... 0. 0. 0.]

cosine_sim[movie_index] similarity_scores =

pd.DataFrame(cosine_sim[movie_index],
columns=["score"])

movie_indices = similarity_scores.sort_values("score",

ascending=False)[1:11].index df['title'].iloc[movie_indices]

9100	Die Konferenz der Tiere	
4653	Little Criminals	
208	Maybe... Maybe Not	
2073	The Golem: How He Came Into the World	
4586	The Way South 883	Pandora
and the Flying Dutchman 1633		Death
in Venice		
4105	Harlan: In the Shadow of Jew Süß	
9254	Island Guests	

Name: title, dtype: object

```
content_based_recommender("Django Unchained", cosine_sim, df)
```

Name: title, dtype: object

```
content based recommender("Cinderella", cosine sim, df)
```

9100	Die Konferenz der Tiere
4653	Little Criminals
208	Maybe... Maybe Not
2073	The Golem: How He Came Into the World
4586	The Way South 883 Pandora
and the Flying Dutchman 1633	Death
in Venice	
4105	Harlan: In the Shadow of Jew Süß
9254	Island Guests
6033	The Obama Deception

Name: title, dtype: object

content_based_recommender("Interstellar", cosine_sim, df)

1930	Saved!
6984	Saturday Night
591	Dry Cleaning
4901	West of Memphis

3112 Diary of the Dead
10859 Origin
1688 Storyville
2556 Stealth
9839 Hide and Seek
11333 American Violence

Name: title, dtype: object

content_based_recommender('Fight Club', cosine_sim, df)

6584 Merlin's Apprentice
8252 The Night Before
7902 After Death 5839 X-
Men: Days of Future Past
10754 Santo en Anónimo mortal
8130 Salem Falls
9598 The Holy Modal Rounders: Bound to Lose
10796 White Coffin
6912 American Heist
3832 Bless the Beasts & Children

Name: title, dtype: object

```
def calculate_cosine_sim(dataframe):  
    tfidf = TfidfVectorizer(stop_words='english')  
    dataframe['overview'] = dataframe['overview'].fillna('')  
    tfidf_matrix = tfidf.fit_transform(dataframe['overview'])  
    cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)  
    return cosine_sim  
  
cosine_sim = calculate_cosine_sim(df)  
content_based_recommender('Fight Club', cosine_sim, df)
```

6584	Merlin's Apprentice	8252
The Night Before		
7902	After Death	5839 X-
Men: Days of Future Past		
10754	Santo en Anónimo mortal	
8130	Salem Falls	
9598	The Holy Modal Rounders: Bound to Lose	
10796	White Coffin	
6912	American Heist	
3832	Bless the Beasts & Children	

Name: title, dtype: object

```
def content_based_recommender_user_input(cosine_sim, dataframe):
```

```
    user_input_movie = input("What movie do you want to watch next? Enter the  
movie title: ")
```

```
    if user_input_movie not in dataframe['title'].values:  
        print("Movie not found in the dataset. Please try another
```

```

movie.")    return

    user_input_rating = input("Enter your desired minimum rating score
(0-10): ")

    try:    user_input_rating = float(user_input_rating)    if
user_input_rating < 0 or user_input_rating > 10:    print("Invalid
rating score. Please enter a number between 0 and 10.")    return
except ValueError:    print("Invalid rating score. Please enter a valid
number between
0 and 10.")    return

    recommendations = content_based_recommender(user_input_movie,
cosine_sim, dataframe, user_input_rating)

    print("\nRecommended movies based on similarity to '{}' with a rating score of {}
or above:".format(user_input_movie, user_input_rating))
    print("-----")
    -----")    for index, movie in
recommendations.iterrows():    print("Title:", movie['title'])
    print("Overview:", movie['overview'])    genres = ',
'.join([genre['name'] for genre in
eval(movie['genres'])])    print("Genres:", genres)
    print("Original Language:", movie['original_language'])    print("Popularity:",
movie['popularity'])
    print("Runtime:", movie['runtime'])    print("Vote Average:",
movie['vote_average'])    print("Vote Count:", movie['vote_count'])
    print("-----")
    -----")

def content_based_recommender(title, cosine_sim, dataframe, min_rating):
indices = pd.Series(dataframe.index, index=dataframe['title'])    indices =
indices[~indices.index.duplicated(keep='last')]    movie_index = indices[title]
    similarity_scores = pd.DataFrame(cosine_sim[movie_index], columns=["score"])
    movie_indices = similarity_scores[(similarity_scores['score'] > 0)]

```


&

```
(dataframe['vote_average'] >=
min_rating)].sort_values("score", ascending=False)[1:11].index
dataframe.loc[movie_indices]
content_based_recommender_user_input(cosine_sim, df)
```

What movie do you want to watch next? Enter the movie title: Django
Unchained

Enter your desired minimum rating score (0-10): 8.5

Recommended movies based on similarity to 'Django Unchained' with a rating
score of 8.5 or above:

Title: My Foolish Heart

Overview: After a long absence, Mary Jane visits her schoolfriend Eloise, and
Eloise's daughter Ramona. Eloise drinks too much and is unhappily married to Lew
Wengler. Eloise falls asleep and remembers her time with her true love, Walt
Dreiser, at the beginning of the Second World War. She recalls the events that lead
up to her split with Mary
Jane, and how Lew married Eloise rather than Mary Jane.

Genres:

Original Language: en

Popularity: 0.065284

Runtime: 98.0

Vote Average: 10.0

Vote Count: 1.0

Title: Recollections of the Yellow House

Overview: In his debut as his libidinous alter ego João de Deus, Monteiro stars as a
meticulous and melancholy bachelor living in a rambling boarding house and
feverishly obsessed with his landlady's daughter. A ribald yet wonderfully delicate
comedy, Memories of the Yellow House traces the indecisive, dangerous ripening
of erotic fantasy as João's voyeuristic rituals push him to further, unexpected
extremes. The rich mellowing of Monteiro's political address is embodied in the

figure of the boarding house whose sanctimonious and narcissistic landlady is both a loose metaphor for the repressive, reactionary tendencies of postwar Portuguese society and an affectionately comic figure.

Genres: Comedy

Original Language: pt

Popularity: 0.002001 Runtime: 122.0

Vote Average: 8.5 Vote

Count: 4.0 -----

Title: Bipolar

Overview: When Harry Poole tries out a new medication for Bipolar Disorder, he is reborn as "Edward Grey", a seductive but dangerous alter ego who dramatically takes over his life, changing the young man and those around him forever.

Genres: Thriller, Mystery

Original Language: en

Popularity: 0.543602

Runtime: 80.0

Vote Average: 9.0

Vote Count: 1.0

Title: Hell to Eternity

Overview: Based on the story about Guy Gabaldon, a Los Angeles Hispanic boy raised in the 1930s by a Japanese-American foster family. After Pearl Harbor, his foster family is interned at the Manzanar camp for Japanese Americans, while he enlists in the Marines, where his ability to speak Japanese becomes a vital asset. During the Battle of Saipan, he convinces 800 Japanese to surrender after their general

commits suicide. Genres: Drama, War

Original Language: en

Popularity: 0.157003

Runtime: 131.0

Vote Average: 10.0

Vote Count: 1.0

Title: Timecode

Overview: Luna and Diego are the parking lot security guards. Diego does the night shift, and Luna works by day.

Genres:

Original Language: en

Popularity: 0.587178 Runtime:

15.0

Vote Average: 8.6

Vote Count: 7.0

Title: Stretch and Bobbito: Radio That Changed Lives

Overview: Exploring the social impact of what the Source Magazine in 1998 voted, "The Best Hip Hop Radio Show Of All-Time." The documentary film is the story of quirky friends who became unlikely legends by engaging their listeners and breaking the biggest rap artists ever.

Genres: Documentary

Original Language: en Popularity:

1.131106 Runtime: 90.0

Vote Average: 9.0

Vote Count: 6.0

Title: Margaret Cho: I'm the One That I Want

Overview: November, 1999, Margaret Cho is home in San Francisco at the Warfield Theater. Cho structures her monologue loosely on her professional life's trajectory: doing stand-up, cast in an ABC-TV sitcom, losing 30 pounds in two weeks for the part, the show's cancellation, a descent into booze, pills, and self-loathing, and a resurrection into her own voice, her own shape, and being the one she wants.

Genres: Comedy

Original Language: en

Popularity: 0.109302

Runtime: 96.0

Vote Average: 8.7

Vote Count: 3.0

Title: Lady Windermere's Fan

Overview: Mrs Erlynne, the mother of Lady Windermere - her daughter does not know about her - wants to be introduced in society, so that she can marry Lord Augustus Lorton. Lord Windermere, who helped her with a cheque, invites her to his wife's birthday-party, but Lady Windermere thinks, she has reason to be jealous, so she decides to leave her husband and go to Lord Darlington, who is pining for her. Mrs Erlynne finds this out and tries to prevent her of this mistake, but her daughter leaves her fan in Lord Darlington's residence.

Genres: Comedy

Original Language: en

Popularity: 0.49755

Runtime: 120.0

Vote Average: 8.5

Vote Count: 2.0

Title: Madagascar

Overview: On the east side of the island, rugged mountains rise dramatically from the palm fringed Indian Ocean. These uplands catch drenching rains almost all year round - steep and inaccessible, they are the most diverse part of the island. "Lost Worlds" travels from the highest mountains, where trees are few and it's cold enough for frost, through the lush, cloaking rainforests, down to the tropical coast, discovering the ringtailed lemurs, the jeweled geckos and the predatory wasps. So what is it that has made this narrow eastern strip in particular so rich in life?

Genres: Documentary, Foreign

Original Language: en

Popularity: 0.15962

Runtime: 240.0

Vote Average: 8.6

Vote Count: 4.0

Title: The Little Hut

Overview: Sir Philip Ashlow (Stewart Granger), his neglected wife, Lady Ashlow (Ava Gardner) and his best friend Henry Brittingham-Brett (David Niven) are shipwrecked on a desert island. This potential ménage à trois where the two men compete for the lady's attention is interrupted by the unexpected arrival of a fourth inhabitant of the island.

Genres: Comedy, Romance

Original Language: en

Popularity: 0.883862

Runtime: 90.0

Vote Average: 10.0

Vote Count: 2.0

```
<ipython-input-148-5c8506a5ba51>:53: UserWarning: Boolean Series key will be
reindexed to match DataFrame index.  movie_indices =
similarity_scores[(similarity_scores['score'] > 0) &
```